

A Practical Guide to Transformer architecture deep learning

TOPIC -- TRANSFORMER ARCHITECTURE DEEP LEARNING

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

-- 01 --

FlashAttention FlashAttention is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow). See the page on softmax for details. An improved version, FlashAttention-2, was developed to cater to the rising demand for language models capable of handling longer context lengths. It offers enhancements in work partitioning and parallelism, enabling it to achieve up to 230 TFLOPs/s on A100 GPUs (FP16/BF16), a 2x speed increase over the original FlashAttention. Key advancements in FlashAttention-2 include the reduction of non-matmul FLOPs, improved parallelism over the sequence length dimension, better work partitioning between GPU warps, and added support for head dimensions up to 256 and multi-query attention (MQA) and grouped-query attention (GQA). Benchmarks revealed FlashAttention-2 to be up to 2x faster than FlashAttention and up to 9x faster than a standard attention implementation in PyTorch. Future developments include optimization for new hardware like H100 GPUs and new data types like FP8. FlashAttention-4 focuses on pipelining to increase instruction throughput, and was developed to perform particularly well on Blackwell GPUs.

-- 02 --

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix M . For example, if the input token's identifier is 3, then the one-hot representation is $[0, 0, 0, 1, 0, 0, \dots]$, and its embedding vector is $\text{Embed}(3) = [0, 0, 0, 1, 0, 0, \dots] M$. The token embedding vectors are added to their

respective positional encoding vectors (see below), producing the sequence of input vectors. The dimension of an embedding vector is called hidden size or embedding size and written as d_{emb} . This size is written as d_{model} in the original transformer paper.

-- 03 --

Efficient implementation The transformer model has been implemented in standard deep learning frameworks such as TensorFlow and PyTorch. Transformers is a library produced by Hugging Face that supplies transformer-based architectures and pretrained models.

-- 04 --

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights W^Q , the key weights W^K , and the value weights W^V . The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length $\ell_{\text{seq, query}}$, and each entry is a vector of dimension $d_{\text{emb, query}}$. Similarly for the key and value sequences. For each vector x_i in the query sequence, it is multiplied by a matrix W^Q to produce a query vector $q_i = x_i W^Q$. The matrix of all query vectors is the query matrix: $Q = X_{\text{query}} W^Q$. Similarly, we construct the key matrix $K = X_{\text{key}} W^K$ and the value matrix $V = X_{\text{value}} W^V$. It is usually the case that all W^Q, W^K, W^V are square matrices, meaning $d_{\text{emb, query}} = d_{\text{query}}$, etc. Attention weights are calculated using the query and key vectors: the attention weight a_{ij} from token i to token j is the dot product between q_i and k_j . The attention weights are divided by the square root of the dimension of the key vectors, $\sqrt{d_k}$, which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that W^Q and W^K are different matrices allows attention to be non-symmetric: if token i attends to token j (i.e. $q_i \cdot k_j$ is large), this does not necessarily mean that

